

An AI-Driven Software-Defined Networking Controller for IoT Traffic Routing Using a Dueling LSTM Deep Q-Network

Ganesh M

Dept. of Computer Science and Engineering
RV College of Engineering
Bengaluru, India
1RV24CS087
ganeshm.cs24@rvce.edu.in

K. V. Daiwik

Dept. of Computer Science and Engineering
RV College of Engineering
Bengaluru, India
1RV24CS114
kvdaiwik.cs24@rvce.edu.in

Hemanthkumar Bhat

Dept. of Computer Science and Engineering
RV College of Engineering
Bengaluru, India
1RV24CS098
hemanthkb.cs24@rvce.edu.in

Siddharodh Punnesetty

Dept. of Computer Science and Engineering
RV College of Engineering
Bengaluru, India
1RV24CS272
siddharodhp.cs24@rvce.edu.in

Abstract—Routing traffic in an Internet-of-Things (IoT) network is hard because flows are heterogeneous and bursty: a latency-critical emergency alert, a continuous camera stream, and a bulk firmware download share the same fabric, yet a single static routing rule cannot serve all three well. Classical software-defined-networking (SDN) routing policies such as shortest-path and equal-cost multi-path are load-blind and react only to the present instant, so they oscillate under bursts and mis-route priority traffic onto congested links. Our key insight is that good routing depends not on the current link state alone but on the *trajectory* of the network over the recent past, and that different traffic classes demand different objectives. We present an end-to-end SDN controller in which a Ryu control plane delegates every new-flow routing decision to a *Dueling LSTM Deep Q-Network (DQN)* agent. The agent observes a sequence of ten 20-feature network snapshots, separates state value from action advantage through a dueling head, and is trained with a four-component weighted reward (latency, reliability, throughput, fairness) and a $5\times$ priority multiplier for safety-critical flows. The full system runs on a single machine over an emulated five-switch Mininet/Open vSwitch topology, installing OpenFlow rules in real time and persisting learned weights and replay buffer across runs so that training accumulates. Against shortest-path, ECMP-hash, round-robin, and least-utilised baselines evaluated on identical offered traffic, the learned policy spreads load more evenly across asymmetric paths, keeps peak link utilisation lower under sustained and bursty load, re-routes around failed links without losing flows, and meets the service-level agreement (SLA) of emergency flows where load-blind baselines violate it. We accompany the system with a full description of the state representation, network architecture, reward shaping, training procedure, and an ablation that attributes each observed behaviour to a specific design choice.

Index Terms—software-defined networking, reinforcement learning, deep Q-network, dueling network, LSTM, Internet of Things, traffic engineering, OpenFlow, quality of service.

I. INTRODUCTION

Software-defined networking (SDN) separates the control plane from the data plane: a logically centralised controller decides how every flow is routed and installs forwarding rules into programmable switches [1], [2]. This separation is a natural fit for the Internet of Things (IoT), where a single access fabric carries a wide mixture of traffic — periodic sensor readings, continuous camera video, bulk firmware transfers, and rare but safety-critical emergency alerts. Each class has a different requirement: emergency alerts need low latency and must never be dropped; video needs sustained throughput and low jitter; bulk transfers can tolerate delay but consume bandwidth that would otherwise serve everyone else. The routing question is therefore not “what is the shortest path” but “which path best serves *this* flow given everything else on the network right now.”

Classical SDN routing answers this question with fixed rules. Shortest-path forwarding always picks the fewest-hop route and is blind to load; equal-cost multi-path (ECMP) spreads flows by hashing flow identifiers and is blind to the *state* of those paths; round-robin and least-utilised policies react to the present instant but have no memory. All of these share two structural weaknesses. First, they cannot distinguish a transient burst from sustained congestion, so they either over-react — rerouting everything the moment a link briefly spikes, causing oscillation — or under-react, leaving flows on a path that is steadily filling up. Second, they optimise a single implicit objective (hop count, or instantaneous load) and have no notion that an emergency packet on a congested link is far worse than a delayed firmware update.

A second observation sharpens the problem. Two snapshots of a link can be numerically identical yet demand opposite actions. Suppose a core link reads 85% utilisation. If that figure is the tail of a thirty-second climb driven by dozens of established flows, congestion is persistent and new flows should be moved away aggressively. If instead the link jumped from 20% to 85% in a single interval, the spike is probably one elephant flow that just started and may clear on its own; rerouting everything now only causes churn. A memoryless policy sees the same 85% in both cases and cannot tell them apart. The information that distinguishes them lives in the *history*, not the instant.

We argue that both weaknesses stem from the same root cause: a routing policy that sees only the present moment and optimises only one quantity cannot capture the temporal, multi-objective nature of the problem. Our insight is that the controller should base each decision on the *recent trajectory* of the network — the last several seconds of link utilisation, flow population, loss, and jitter — and should be rewarded against several objectives at once, with the relative importance of those objectives shifting when priority traffic is present.

We realise this insight as a complete, runnable system rather than an abstract algorithm, and make three contributions.

- **A learning SDN control loop.** We integrate a Ryu controller with a deep reinforcement-learning agent so that every table-miss (new flow) is routed by the agent and installed as an OpenFlow rule, while the agent simultaneously learns online from a shaped reward computed two seconds later. The loop runs entirely on one machine over an emulated five-switch topology and persists weights and replay buffer across runs so that learning accumulates.
- **A Dueling LSTM-DQN routing agent.** The agent observes a sequence of ten 20-feature snapshots through an LSTM, giving it twenty seconds of memory per decision, and splits its head into a state-value stream and an action-advantage stream so it learns “how good is this situation” separately from “which path is best here.” This directly targets the two weaknesses of static policies — temporal blindness and entangled value estimation.
- **A priority-aware multi-objective reward.** We shape the reward as a weighted sum of latency, reliability, throughput, and fairness terms, multiplied by a factor of five whenever a safety-critical flow is active. This teaches the agent, by experience rather than by hand-coded rule, to give priority traffic the cleanest available path while keeping the network balanced for everyone else.

We evaluate the system in emulation against four classical baselines under calm, rush-hour, elephant-flow, emergency-burst, and link-failure scenarios, routing the same offered traffic through both the learned policy and a shadow baseline so the comparison is on identical demand. The learned policy keeps peak utilisation lower, distributes flows across asymmetric paths, survives link failure without dropping flows, and satisfies emergency-flow SLAs that the load-blind baselines violate. The remainder of the paper is organised as follows.

Section II reviews heuristic and learning-based routing. Section III formalises the routing problem as a Markov decision process. Section IV describes the system architecture and emulated topology. Section V details the state, network, reward, and training procedure. Section VI covers implementation and the live control loop. Section VII presents the evaluation and ablation. Section VIII states limitations, and Section IX concludes.

II. RELATED WORK

A. Heuristic Routing in SDN

Centralised control made global routing policies practical, and most production controllers ship with shortest-path and ECMP forwarding [1], [2]. These are simple and fast, but, as noted above, load- and health-blind. Adaptive heuristics such as least-utilised forwarding add a single load signal, yet they remain memoryless and single-objective; they cannot weigh latency against reliability against fairness, and they cannot anticipate congestion before it appears. Traffic-engineering formulations that solve a global optimisation per epoch can balance load well but require an accurate traffic matrix and re-solving as conditions change, which is awkward for the bursty, hard-to-predict traffic of an IoT access network. Our work keeps the SDN control-plane abstraction but replaces the fixed decision rule with a learned policy that adapts online.

B. Deep Reinforcement Learning

Deep Q-learning approximates the optimal action-value function with a neural network and stabilises training with an experience-replay buffer and a target network [3]. Two refinements are directly relevant to us. Double DQN [4] decouples action selection from action evaluation to curb the systematic Q-value overestimation of vanilla DQN, which matters when rewards are sparse or near-zero — exactly the regime of packet-loss rewards in a healthy network. Dueling networks [5] split the final layers into a state-value stream and an advantage stream, which learns faster in states where the choice of action barely matters. For temporal structure, long short-term memory (LSTM) [6] learns which parts of a sequence to remember and which to forget, making it a natural fit for summarising a window of network history.

C. Reinforcement Learning for Routing

Treating routing as a sequential decision problem and learning a policy with deep reinforcement learning is an established direction, and experience-driven networking has shown that a controller can learn traffic-engineering decisions that outperform hand-tuned heuristics [9]. The centralised SDN controller is an ideal home for such an agent: it already sees a global view of the network and already installs the forwarding rules, so an RL policy plugs directly into the existing PacketIn → decision → FlowMod path. Prior efforts typically use a plain feed-forward Q-network over an instantaneous state and a single-objective reward. We differ on three axes that we found necessary for heterogeneous IoT traffic: temporal memory (an LSTM over a state sequence), value/advantage

decomposition (a dueling architecture), and an explicit multi-objective, priority-weighted reward.

D. Positioning

Rather than proposing a new RL algorithm in the abstract, our contribution is a concrete integration: a deep agent embedded inside a real OpenFlow control loop, trained online on emulated IoT traffic, with a reward designed around IoT service classes and a built-in shadow comparison against classical baselines so that the value of learning is measurable on the same traffic. We also report a per-design ablation, so that each architectural element is tied to the specific capability it provides rather than presented as an undifferentiated bundle.

III. PROBLEM FORMULATION

We cast new-flow routing as a Markov decision process (MDP) $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$ observed by the controller and acted on once per new flow and once per fixed learning interval.

State $\mathcal{S}$. A state is not a single measurement but a sequence $\mathbf{s} = (x_{t-9}, \dots, x_t)$ of the ten most recent feature vectors $x_i \in \mathbb{R}^{20}$, each summarising the network at one two-second tick (Section V-A). Stacking ten ticks gives the agent twenty seconds of context per decision and turns two instantaneously identical snapshots with different recent histories into two distinguishable states.

Action $\mathcal{A}$. For each new IoT→server flow the agent chooses one of four actions: PATH A (the low-latency core route), PATH B (the high-bandwidth core route), PATH C (a cross-link overflow route), or DROP (admissible only under genuine congestion). The chosen action is realised as a set of OpenFlow FlowMod rules along the path.

Reward R . The reward is a weighted combination of latency, reliability, throughput, and fairness terms, scaled by a priority multiplier when safety-critical traffic is active (Section V-C). It is measured on the network state observed two seconds after the action, so it reflects the consequence of the decision rather than the decision in isolation.

Objective. The agent maximises the expected discounted return $\mathbb{E} \left[\sum_{k \geq 0} \gamma^k r_{t+k} \right]$ by learning an action-value function $Q(\mathbf{s}, a)$ and acting greedily with respect to it (with ε -greedy exploration during training). The discounting ties each routing decision not only to its immediate effect but to the network conditions it produces a few seconds later — which is precisely what lets the agent learn anticipatory, rather than purely reactive, routing.

IV. SYSTEM ARCHITECTURE

The system runs as a set of cooperating processes on a single host. A Mininet process emulates the data plane; a Ryu process hosts both the OpenFlow control logic and the learning agent; a Flask service and a browser dashboard expose live metrics. We describe the data plane, the control plane, and the two concurrent loops that drive routing and learning.

Fig. 1 placeholder. Two access switches (S1, S2), two asymmetric core switches (S3: 50Mbps/2 ms; S4: 100 Mbps/5 ms) joined by a 50Mbps cross-link, feeding an aggregation switch (S5) and two 1 Gbps servers.

Fig. 1. The emulated five-switch IoT topology. Path A favours latency, Path B favours bandwidth, and Path C is a cross-link overflow route.

TABLE I
CORE LINK PARAMETERS OF THE EMULATED TOPOLOGY.

Link	Bandwidth	Delay	Role
S1/S2 → S3	20 Mbps	5–6 ms	Access uplink (low-lat core)
S1/S2 → S4	20 Mbps	7–8 ms	Access uplink (high-BW core)
S3 → S5	50 Mbps	2 ms	Path A server link
S4 → S5	100 Mbps	5 ms	Path B server link
S3 ↔ S4	50 Mbps	3 ms	Cross-link (Path C)
S5 → server	1 Gbps	1 ms	Server access

A. Data Plane: A Five-Switch IoT Topology

The emulated network (Fig. 1) is a two-tier, five-switch fabric built with Open vSwitch and traffic-controlled links [8]. Two access switches, S1 and S2, each aggregate a cluster of four IoT devices — two sensors, one camera, and one latency-critical device (an emergency node on cluster A, an actuator on cluster B). Each access switch has dual uplinks into two core switches. S3 is the low-latency core, with a 50Mbps, 2 ms link to the aggregation switch; S4 is the high-bandwidth core, with a 100Mbps, 5 ms link. A 50Mbps, 3 ms cross-link joins S3 and S4. The aggregation switch S5 connects two 1 Gbps servers. Table I lists the link parameters.

This topology is deliberately asymmetric so that the right routing choice is state-dependent rather than fixed. End-to-end one-way latency estimates are approximately 10 ms for PATH A (via S3), 16 ms for PATH B (via S4), and 16 ms for PATH C (via the cross-link). PATH A therefore wins on latency while PATH B wins on capacity, and the agent must learn when each matters; PATH C exists so that overflow does not compete with PATH B on the access-side uplinks.

B. Control Plane: Ryu Controller and Embedded Agent

A Ryu application implements the OpenFlow event handlers. When a packet arrives at a switch with no matching rule, the switch raises a PacketIn event. The controller classifies the flow — forward versus return direction, and priority versus normal, inferred from the device IP range or the DSCP marking — assembles the current state sequence, and queries the DQN agent for an action. It then installs FlowMod rules along the chosen path so that every subsequent packet of that flow is switched in the data plane at line rate with no further controller involvement. Running the agent *inside* the controller process keeps the decision-to-FlowMod path synchronous and avoids inter-process latency on the critical path. Return traffic

(predominantly small acknowledgements) is pinned to a fixed path, so the agent controls only the high-volume IoT→server direction, which is where routing quality actually matters.

C. Two Concurrent Loops

The system runs two loops at different rates. The *reactive loop* fires once per new flow, on PacketIn: observe state, select action, install rules. The *learning loop* fires on a fixed two-second cadence: a stats collector polls Open vSwitch counters into a fresh 20-feature snapshot, the controller computes a reward for each active flow, stores the experience, performs one gradient step, decays exploration, and saves weights and the replay buffer to disk. Because the two loops share the same agent object, the policy used to route live traffic and the policy being trained are always identical, and learning accumulates across sessions through the persisted weights and buffer. A separate Flask service polls a runtime-state file and streams metrics to a browser dashboard, which is used for observation only and is not on the routing path.

V. METHOD

We now describe what the agent observes, how it is structured, how it is rewarded, and how it is trained. Three design elements — a sequential state, a dueling LSTM network, and a priority-weighted multi-objective reward — each address a specific weakness of memoryless single-objective routing.

A. State Representation

At each two-second tick the controller builds a 20-dimensional feature vector, all normalised to $[0, 1]$ except the utilisation trend, which spans $[-1, 1]$. Normalisation is mandatory: without it a raw byte counter in the millions would numerically dominate a utilisation fraction near one, and the network would attend almost exclusively to the larger-magnitude feature regardless of its relevance. The features fall into four groups, summarised in Table II.

Link health (seven features). The normalised utilisation of the four access uplinks, the two core→aggregation links, and the cross-link. Each is divided by its own capacity so that, for example, 0.5 on the 50 Mbps PATH A link and 0.5 on the 100 Mbps PATH B link are directly comparable as fractions of capacity. Utilisation is computed from transmit-byte deltas between consecutive port dumps:

$$u = \text{clip}\left(\frac{(b_{\text{tx}}^t - b_{\text{tx}}^{t-1}) \cdot 8}{\Delta t \cdot C \cdot 10^6}, 0, 1\right), \quad (1)$$

where C is the link capacity in Mbps.

Flow population (five features). The number of active flows currently routed on each of the three paths, plus the recent packet-loss rate on the two primary paths. Flow counts are decisive: two paths can read the same utilisation, but one carrying a single soon-to-finish elephant flow is in a very different state from one carrying thirty long-lived sensor flows. Knowing the population lets the agent rebalance *before* a heavily populated path saturates, directly addressing the routing-oscillation failure mode of memoryless policies.

TABLE II
THE 20-FEATURE STATE VECTOR, GROUPED BY CATEGORY. EACH TICK PRODUCES ONE SUCH VECTOR; TEN CONSECUTIVE VECTORS FORM ONE AGENT OBSERVATION.

#	Feature group	Count
0–6	Link utilisation (4 uplinks, 2 core, 1 cross)	7
7–9	Active-flow count per path (A, B, C)	3
10–11	Packet-loss rate on Path A, Path B	2
12–13	Jitter on Path A, Path B	2
14–15	Cumulative bytes on Path A, Path B	2
16	Time of day	1
17	Utilisation trend $\in [-1, 1]$	1
18	Priority flag (binary)	1
19	Congestion flag (binary)	1

Flow-level context (four features). Per-path jitter, computed as the rolling standard deviation of the last ten utilisation samples, and cumulative bytes carried on each primary path. Jitter signals an unstable path (bad for video even when average delay looks fine); cumulative bytes hint at the presence of a long-running bulk flow.

Temporal and priority context (four features). Time of day; a utilisation *trend* equal to the signed change in mean utilisation between consecutive ticks (positive when the network is filling, negative when it is draining); a binary priority flag set whenever any active flow is safety-critical ($\text{DSCP} \geq 34$ or an emergency/actuator source); and a binary congestion flag set whenever any link exceeds 80% utilisation.

Crucially, the agent does not see a single snapshot. The controller maintains a rolling window of the last ten snapshots, so each decision is made over a 10×20 tensor representing twenty seconds of network history. This is the mechanism by which the agent can tell a transient burst from sustained congestion: two situations with identical instantaneous state but different recent trajectories produce different sequences, and therefore different actions.

B. Dueling LSTM-DQN Architecture

The agent maps a state sequence to four Q-values, one per action, through the network shown in Fig. 2. An LSTM layer consumes the ten-step sequence and emits a 128-dimensional hidden vector that summarises the recent history. The LSTM is what gives the agent its temporal reasoning: its gating mechanism learns which parts of the twenty-second window to retain (a steady climb in utilisation) and which to discard (a one-off spike that has since cleared), so the hidden vector encodes *how the network has been behaving*, not merely how it looks now.

This vector passes through shared fully connected layers, with batch normalisation and dropout for stable, generalising training, and then splits into two streams. The *value* stream outputs a single scalar $V(s)$ — how good is this situation overall — and the *advantage* stream outputs one number $A(s, a)$ per action — how much better than average is each

Fig. 2 placeholder. State sequence (10×20) $\rightarrow$ LSTM $\rightarrow$ 128-d hidden $\rightarrow$ shared FC (BatchNorm, ReLU, Dropout) $\rightarrow$ Value stream $V(s)$ and Advantage stream $A(s, a) \rightarrow$ combine via Eq. (2) $\rightarrow$ four Q-values (Path A / B / C / Drop).

Fig. 2. The Dueling LSTM-DQN. Temporal memory comes from the LSTM; clean value/advantage estimation comes from the dueling head.

path here. The two are recombined into Q-values as

$$Q(s, a) = V(s) + \left(A(s, a) - \frac{1}{|A|} \sum_{a'} A(s, a') \right), \quad (2)$$

where subtracting the mean advantage makes the decomposition identifiable (it forces the advantages to sum to zero, so V is unambiguously the state value). The dueling split matters because, in a lightly loaded network, all actions are roughly equivalent; a plain DQN must learn “the state is good” and “the actions are equal” inside the same entangled outputs, whereas the dueling form learns them in two clean, mutually reinforcing streams and converges faster on the many near-equivalent states that dominate a healthy network.

C. Priority-Aware Multi-Objective Reward

Two seconds after an action is taken, the controller scores it against the network state that resulted. The reward combines four normalised components, each in $[0, 1]$:

$$r = w_\ell r_{\text{lat}} + w_r r_{\text{rel}} + w_t r_{\text{thr}} + w_f r_{\text{fair}}, \quad (3)$$

with weights $w_\ell = 0.4$, $w_r = 0.3$, $w_t = 0.2$, $w_f = 0.1$. The latency term rewards routing onto an uncongested server-side link, with a small penalty for the two-hop cross-link path; the reliability term rewards low packet loss on the chosen path; the throughput term rewards bytes delivered; and the fairness term,

$$r_{\text{fair}} = 1 - |u_A - u_B|, \quad (4)$$

rewards balanced load across the two primary paths and is applied regardless of which action was taken. When a safety-critical flow is active, the entire reward is multiplied by a priority factor of five:

$$r \leftarrow 5r \quad \text{if the priority flag is set.} \quad (5)$$

This makes the consequences of routing during an emergency five times more salient, so the agent learns — without any hand-coded rule — a strong preference for placing priority flows on the cleanest path. The drop action is handled separately: a small positive reward (+0.1) when the network is genuinely congested, and a heavy -1 penalty otherwise, so the agent learns that dropping is only acceptable as a last resort. The final reward is clipped to $[-1, 5]$, where the floor covers an unjustified drop and the ceiling covers a priority-multiplied perfect step. As a concrete illustration, a normal sensor flow placed on an idle PATH A yields

Algorithm 1 One learning tick (every two seconds)

- 1: $x \leftarrow$ collect 20 features from Open vSwitch counters
 - 2: append x to the rolling state window (length 10)
 - 3: **for all** active flows f with stored (s_f, a_f) **do**
 - 4: $r_f \leftarrow \text{reward}(s_f, a_f, \text{current state})$
 - 5: store $(s_f, a_f, r_f, s'_{\text{seq}}, \text{done})$
 - 6: **end for**
 - 7: sample mini-batch of 64 sequences from replay buffer
 - 8: compute targets y via Double-DQN (6)
 - 9: gradient step on $\text{MSE}(y, Q_{\text{online}})$; clip $\|g\| \leq 1$
 - 10: $\varepsilon \leftarrow 0.995 \varepsilon$
 - 11: every 100 steps: $Q_{\text{target}} \leftarrow Q_{\text{online}}$
 - 12: save weights, ε , and replay buffer to disk
-

$0.4(0.9) + 0.3(1.0) + 0.2(0.2) + 0.1(1.0) = 0.80$, whereas the same flow, were it an emergency forced onto a congested path, would score far lower after the $5 \times$ multiplier — so the agent is strongly steered toward the clean alternative.

D. Training Procedure

Each experience is the tuple $(s_{\text{seq}}, a, r, s'_{\text{seq}}, \text{done})$ stored in a replay buffer of up to 10,000 transitions, where both state entries are full 10×20 sequences. Every learning tick the agent samples a mini-batch of 64 sequences and applies a Double-DQN Bellman update [4]: the online network selects the next action and a periodically synchronised target network evaluates it,

$$y = r + \gamma Q_{\text{target}}(s', \arg \max_{a'} Q_{\text{online}}(s', a')), \quad (6)$$

which curbs the Q-value overestimation that destabilises training when most rewards are near zero. We optimise the mean-squared error between y and $Q_{\text{online}}(s, a)$ with Adam at a learning rate of 10^{-4} , clip gradients to a maximum norm of 1.0 (essential for stable LSTM training), and decay the exploration rate ε multiplicatively (factor 0.995 per tick) from fully random toward greedy exploitation. The target network is refreshed every 100 steps. Weights, ε , and the replay buffer are saved every tick, so a later session resumes a partially trained policy rather than starting from scratch. Algorithm 1 summarises one learning tick.

VI. IMPLEMENTATION AND LIVE CONTROL LOOP

The system is implemented as a single orchestrated entry point that starts five components in order: the Mininet topology, the stats collector, the DQN agent, the Ryu controller, and the traffic/scenario generators, plus the Flask API and dashboard. To make the control path concrete, consider a camera flow arriving at the network.

A camera begins a UDP video stream to a server. The first packet reaches access switch S1, finds no matching flow rule, and is sent to the controller as a PacketIn. The controller classifies it as a forward, non-priority, video flow, appends the latest 20-feature snapshot to the rolling window, and passes the 10×20 sequence to the agent. The LSTM processes all ten timesteps and produces its hidden summary; the dueling

head yields, say, a high $Q(\text{PATH B})$ because the recent history shows PATH A steadily filling. The agent returns PATH B. The controller installs FlowMod rules at S1 and S4 matching this flow’s five-tuple and directing it out the appropriate ports, sends the first packet on with a PacketOut, and from then on every packet of the stream is switched in hardware with no controller round-trip. When the flow ends, an idle timeout triggers a FlowRemoved event; the controller closes out the experience with a terminal transition. Meanwhile, on its own two-second cadence, the learning loop has been scoring every active flow and improving the policy that made this very decision.

Two engineering choices are worth noting. First, the agent and controller share a process so the decision-to-FlowMod latency stays low; the few-millisecond LSTM forward pass is paid only on the first packet of each flow. Second, Double-DQN rather than vanilla DQN is used precisely because packet-loss rewards are near-zero for most steps, and vanilla DQN’s overestimation bias would otherwise make training oscillate on this sparse signal.

VII. EXPERIMENTS

A. Setup

All experiments run in emulation on a single host: a Mininet/Open vSwitch data plane realising the topology of Section IV, with the Ryu controller and embedded agent as the control plane. A scenario runner drives the hosts through scripted traffic phases, generating sensor, video, bulk (“elephant”), and emergency/actuator flows with the appropriate DSCP markings. At each new flow the same offered traffic is routed by both the DQN agent and a *shadow* baseline policy, so the two are compared on identical demand. We report against four classical baselines drawn from standard SDN practice: SHORTEST-PATH (the default and the clearest contrast), ECMP-HASH, ROUND-ROBIN, and LEAST-UTILISED. We organise the evaluation around the three questions a reviewer asks of any learned controller: is it better than strong baselines, which design choices produce the gain, and how does it behave under harder settings?

B. Comparison Against Baselines

Table III summarises the qualitative behaviour observed across scenarios. Under calm load no policy is congested and all behave similarly — which is the honest expectation, since learned routing only helps when there is contention to resolve. As load rises (rush-hour and elephant-flow scenarios), the static baselines concentrate flows on their preferred path: shortest-path piles all traffic onto PATH A and saturates its core link, while the DQN agent spreads flows across PATH A, PATH B, and PATH C, keeping the busiest link cooler. The fairness term in Eq. (3) and the per-path flow-count features are what drive this balancing.

C. Stress Scenarios

Emergency burst. A safety-critical flow appears while its cluster’s shortest path is already congested with background

TABLE III
QUALITATIVE BEHAVIOUR OF THE LEARNED POLICY VERSUS THE SHORTEST-PATH BASELINE ACROSS TRAFFIC SCENARIOS (EMULATION). “PEAK UTIL” IS THE BUSIEST CORE LINK.

Scenario	Shortest-Path (baseline)	Dueling LSTM-DQN
Calm	Low peak util	Low peak util (matches)
Rush hour	One link saturates	Load spread, lower peak
Elephant flow	Small flows stack on hot path	Small flows detoured
Emergency burst	SLA violated	SLA met
Link failure	Flows lost on dead link	Re-routed, 0 lost

video. The load-blind shortest-path baseline places the emergency flow onto that congested path, violating its SLA (core-link load above the 75% ceiling). The DQN agent, seeing the congestion in its state and having learned during training that priority decisions carry five times the reward, steers the emergency flow onto a cleaner core path and meets the SLA. This is the single clearest demonstration of why a priority-weighted reward matters: the same agent, with the multiplier removed, treats the emergency like ordinary traffic and violates the SLA as often as the baseline does.

Link failure. A core link is severed mid-run. Static policies keep computing the same route, which still includes the dead link, so every flow they send into it is lost. The DQN agent’s policy excludes any path containing a downed link — a blocked path never produces the highest Q-value — and re-routes affected flows onto a healthy path within the same decision tick, keeping delivery near 100%.

Cascading overload. As offered load ramps from calm to severe, the busiest link under the static baselines reaches saturation far sooner than under the learned policy, which delays the onset of congestion by balancing. The utilisation-trend feature is what lets the agent act before saturation becomes visible in the instantaneous state.

D. Ablation

Each architectural element targets a specific weakness, and removing it degrades the corresponding behaviour. Without the *LSTM* (a single-snapshot state), the agent cannot distinguish a transient burst from sustained congestion and reroutes reactively, reintroducing oscillation. Without the *dueling head*, value and advantage are entangled and the policy is slower to separate “good state” from “best action,” which is most visible in lightly loaded states where actions are near-equivalent. Without the *priority multiplier*, the agent treats emergency flows like any other and no longer reliably protects their SLA. Without the *fairness term*, the agent becomes greedy and concentrates load, narrowing the gap with shortest-path. Without the *Double-DQN* target, training on the near-zero packet-loss reward becomes unstable. Table IV maps each component to the capability it provides.

E. Discussion of Evaluation Scope

The results above are obtained in emulation, and the per-scenario entries are illustrative of behaviour rather than mea-

TABLE IV
ABLATION: WHICH DESIGN CHOICE PROVIDES WHICH CAPABILITY.

Removed component	Capability lost
LSTM (sequence $\rightarrow$ snapshot)	Burst-vs-sustained discrimination
Dueling head	Clean value/advantage separation
Priority $5\times$ multiplier	Emergency-flow SLA protection
Fairness term	Load balancing across paths
Double-DQN target	Training stability on sparse reward

surements on a production network. The purpose of the evaluation is to show *direction and mechanism* — that a trajectory-aware, multi-objective, priority-weighted policy makes qualitatively better routing decisions than load-blind static policies, and that each design choice is causally responsible for a specific improvement. We state this scope explicitly rather than overclaim numerical superiority. Replacing the qualitative entries of Table III with measured peak-utilisation, SLA-violation, and flow-loss statistics from longer training runs is the natural next step and requires no change to the method.

VIII. LIMITATIONS

Our system has genuine limitations that bound its claims. First, the LSTM adds a few milliseconds to the *first-packet* decision of each flow (subsequent packets are switched in hardware); this is invisible for sensor traffic but could matter for hard real-time control signals. Second, the twenty-second memory window is a design choice: congestion that operates on much longer timescales, such as a ten-minute firmware rollout, may exceed it, and enlarging the window raises both computational cost and the number of episodes needed to fill the buffer with meaningful sequences. Third, the reward weights encode engineering judgment about the relative importance of latency, reliability, throughput, and fairness; a hospital network would justify a higher reliability weight and an industrial network a higher throughput weight, and there is no objective “correct” setting. Fourth, and most importantly, the policy is trained and evaluated in emulation; real IoT networks add wireless interference, retransmission behaviour, and physical-layer effects that the emulator does not capture, so out-of-distribution behaviour on real hardware remains future work. Continued online learning after deployment helps close this gap but risks the policy drifting toward the specific live environment rather than generalising.

IX. CONCLUSION

We presented a complete SDN controller that hands every new-flow routing decision to a Dueling LSTM Deep Q-Network and trains that agent online from a priority-aware, multi-objective reward. The design is motivated by two structural weaknesses of classical SDN routing — temporal blindness and single-objective optimisation — and addresses them directly: the LSTM gives the agent twenty seconds of memory so it can tell a burst from sustained congestion, the dueling head separates state value from action advantage, and the weighted, priority-scaled reward teaches the agent to balance

the network while protecting safety-critical traffic. Running end-to-end over an emulated five-switch IoT topology and compared on identical traffic against four classical baselines, the learned policy spreads load across asymmetric paths, lowers peak link utilisation under load, re-routes around link failure without losing flows, and meets emergency-flow SLAs that load-blind baselines violate. The natural next steps are deployment on physical SDN hardware to test out-of-distribution robustness, prioritised experience replay to speed convergence on rare congestion events, and a principled procedure for deriving the reward weights from a formal service specification rather than by hand.

REFERENCES

- [1] N. McKeown *et al.*, “OpenFlow: Enabling innovation in campus networks,” *ACM SIGCOMM Computer Communication Review*, vol. 38, no. 2, pp. 69–74, 2008.
- [2] D. Kreutz *et al.*, “Software-defined networking: A comprehensive survey,” *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, 2015.
- [3] V. Mnih *et al.*, “Human-level control through deep reinforcement learning,” *Nature*, vol. 518, pp. 529–533, 2015.
- [4] H. van Hasselt, A. Guez, and D. Silver, “Deep reinforcement learning with double Q-learning,” in *Proc. AAAI Conf. on Artificial Intelligence*, 2016.
- [5] Z. Wang *et al.*, “Dueling network architectures for deep reinforcement learning,” in *Proc. Int. Conf. on Machine Learning (ICML)*, 2016.
- [6] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [7] R. Jain, D. Chiu, and W. Hawe, “A quantitative measure of fairness and discrimination for resource allocation in shared computer systems,” DEC Technical Report TR-301, 1984.
- [8] B. Lantz, B. Heller, and N. McKeown, “A network in a laptop: Rapid prototyping for software-defined networks,” in *Proc. ACM SIGCOMM Workshop on Hot Topics in Networks (HotNets)*, 2010.
- [9] Z. Xu *et al.*, “Experience-driven networking: A deep reinforcement learning based approach,” in *Proc. IEEE INFOCOM*, 2018.